



Test Technique

Sujet : StockFlow – Gestion Stock & Commandes (Mini ERP)

Objectif

Développer une application web pour une petite entreprise afin de gérer :

- **Produits**
- **Catégories**
- **Clients**
- **Commandes**
- **Stock (entrées/sorties)**
- **Dashboard** simple (statistiques)

1) Livrables attendus

1. **Conception DB obligatoire**

- ERD (schéma) + relations + contraintes

2. Backend Node.js + Express

- API REST + validation + JWT + rôles

3. Frontend React

- Pages CRUD + affichage commandes + dashboard

4. PostgreSQL

- Script SQL ou migrations

5. README

- Installation + variables d'environnement + commandes
-

2) Contraintes techniques

Backend

- Express.js + Node.js
- PostgreSQL (SQL natif ou ORM autorisé)
- JWT + bcrypt
- Validation (Zod/Joi)
- Gestion d'erreurs propre

Frontend

- React (Vite recommandé)
 - UI simple (Tailwind facultatif)
 - Appels API REST
-

3) Modules & fonctionnalités

3.1 Authentification & rôles

- `admin` : accès total
- `staff` : gestion commandes + stock (pas suppression produits)

Endpoints minimum :

- `POST /api/auth/login`
 - `GET /api/auth/me`
-

3.2 Catégories

Champs minimum :

- `id`, `name` (unique), `createdAt`

Fonctions :

- CRUD complet (admin)
 - Liste catégories (staff)
-

3.3 Produits

Champs minimum :

- `id`
- `name`
- `sku` (unique)
- `price`
- `categoryId`
- `stockQuantity` (calculable ou stocké)
- `createdAt`

Fonctions :

- CRUD
 - Recherche `name` / `sku`
 - Pagination
-

3.4 Clients

Champs minimum :

- `id`
- `fullName`

- `phone`
- `email` (optionnel)
- `createdAt`

Fonctions :

- CRUD
 - Rechercher par nom / téléphone
-

3.5 Commandes

Une commande contient :

- `id`
- `clientId`
- `status` : `draft`, `confirmed`, `cancelled`, `delivered`
- `totalAmount`
- `createdBy`
- `createdAt`

Une commande doit avoir des **lignes de commande** :

- produit
- quantité
- prix unitaire
- sous-total

✅ Règles métier obligatoires :

1. Une commande **confirmed** ne peut pas être modifiée (lignes verrouillées)
 2. Une commande **confirmed** doit vérifier que le stock est suffisant
 3. Quand une commande passe à **confirmed**, le stock diminue automatiquement
-

3.6 Stock (Entrées / Sorties)

Créer un système de mouvements de stock :

- `type` : `IN` / `OUT`
- `productId`
- `quantity`
- `reason`

- `createdBy`
- `createdAt`

✓ Règle obligatoire :

- Interdire un stock négatif (pas de OUT si stock insuffisant)
-

4) Dashboard (obligatoire)

Afficher au minimum :

- Nombre total produits
 - Nombre total clients
 - Commandes du jour
 - Top 5 produits les plus vendus (quantité)
-

5) Conception Base de Données (obligatoire)

Le candidat doit livrer :

- un schéma complet (ERD)
- tables minimales recommandées :

Tables attendues

- `users`
- `categories`
- `products`
- `clients`
- `orders`
- `order_items`
- `stock_movements`

Contraintes attendues

- `sku` UNIQUE
- `category.name` UNIQUE
- relations FK cohérentes

- index sur : `sku` , `name` , `createdAt` , `status`
-

6) API REST minimum attendue

Produits

- `POST /api/products`
- `GET /api/products?search=&page=&limit=`
- `GET /api/products/:id`
- `PUT /api/products/:id`
- `DELETE /api/products/:id` (admin)

Clients

- `POST /api/clients`
- `GET /api/clients?search=`
- `PUT /api/clients/:id`

Commandes

- `POST /api/orders`
- `POST /api/orders/:id/items`
- `PATCH /api/orders/:id/status`
- `GET /api/orders?status=&date=`

Stock

- `POST /api/stock-movements`
 - `GET /api/stock-movements?productId=`
-

7) Frontend (obligatoire)

Pages minimum :

- Login
- Dashboard
- Produits (liste + add/edit + search + pagination)

- Clients (liste + add/edit)
 - Commandes (créer commande + ajouter produits + changer status)
 - Stock mouvements (ajouter + historique)
-

8) Données de test (seed)

Inclure un seed avec :

- 1 admin + 2 staff
 - 5 catégories
 - 15 produits
 - 5 clients
 - 10 commandes (différents statuts)
 - mouvements stock IN/OUT
-

9) Bonus (facultatif)

- Docker Compose (api + db + front)
- Swagger/OpenAPI
- Tests backend (Jest/Supertest)
- Export PDF/CSV d'une commande